SLIDE 1 OF 4

System Architecture & Core Data Structures

C++ CPU Scheduling Simulator Overview

Project Goal & System Overview

This program is an interactive **CPU Scheduling Simulator** written in standard C++. It models how an Operating System manages process execution across various scheduling algorithms. It dynamically takes process metadata, executes five core algorithms, computes standard performance metrics, and automatically launches an HTML-based Graphical User Interface (GUI).

1. Process Entity Representation

Defined via `struct Process` to maintain process lifecycle metadata:

- `id`, `arrivalTime`, `burstTime`: Basic inputs.
- `remainingTime`: Critical for preemptive tracking.
- `priority`: Used for priority-based scheduling.
- `completionTime`, `turnaroundTime`, `waitingTime`: Calculated output performance indicators.

2. Custom Container Implementations

Rather than relying entirely on standard library containers, the program implements custom templates:

- `LinkedList<T>`: Custom singly-linked list managing dynamic memory allocations.
- `Queue<T>` & `Stack<T>`: Wrapper classes built over the custom linked list for FIFO/LIFO operations.
- `GanttBlock`: Struct storing execution history slices for Gantt chart generation.

Key Highlight: Custom Priority Queue

The algorithm engine relies heavily on a specialized `PriorityQueue` class built as a sorted linked list. It dynamically sorts elements during `enqueue()` based on two modes:

Mode	Target Algorithms	Ordering Criteria
`Mode 1`	SJF / SRTF	Sorts by smallest `remainingTime` (FCFS break-tie by `arrivalTime`)
`Mode 2`	Priority Scheduling	Sorts by lowest numerical `priority` value (Higher priority)

SLIDE 2 OF 4

Scheduling Algorithms (Part 1)

Non-Preemptive & Preemptive Shortest-Job Strategies

1. First Come First Serve (FCFS)

A simple non-preemptive algorithm that executes processes in order of their arrival time.

- **Implementation:** First sorts the processes array by `arrivalTime` using Bubble Sort.
- **Execution:** Iterates through sorted processes, advancing CPU clock time by the process's full `burstTime`.
- **Calculations:** `Completion = currentTime`, `Turnaround = Completion - Arrival`, `Waiting = Turnaround - Burst`.

2. Shortest Job First (SJF - Non-Preemptive)

Selects the process with the smallest total burst time among all arrived processes.

- At each step, scans arrived processes and pushes them into `PriorityQueue(mode=1)`.
- Pops the top process (shortest job) and runs it to completion without interruption.
- Handles CPU idle state by advancing `currentTime` if no process has arrived.

3. Shortest Remaining Time First (SRTF - Preemptive SJF)

Preemptive version of SJF where execution happens unit-by-unit (1 ms ticks).

- At every time unit, re-evaluates all active processes using `PriorityQueue(mode=1)`.
- If a newly arrived process has shorter remaining time, the current process is preempted.
- Merges contiguous time slices of the same process into single `GanttBlock` entries for neat chart display.

Code Snippet Focus: SRTF Preemption Mechanism

```
        // Process execution tick in SRTF Process currentP = pq.top(); // Get process with
```

SLIDE 3 OF 4

Scheduling Algorithms (Part 2) & System Utilities

Priority, Round Robin, and Helper Functions

4. Priority Scheduling (Non-Preemptive)

- Executes processes based on explicit priority numbers.
- **Convention:** Lower integer value indicates higher priority (e.g., Priority 1 > Priority 3).
- Uses `PriorityQueue(mode=2)` to select the highest priority available process whenever CPU becomes idle.

5. Round Robin (RR)

- Designed for time-sharing systems using fixed **Time Quantum** slices.
- Uses custom `Queue<int>` to manage ready processes.
- Executes process for `min(quantum, remainingTime)`.
- Re-enqueues incomplete processes after pushing newly arrived ones into the ready queue.

Key Helper & Utility Functions

Function	Role & Description
`resetProcesses()`	Resets process remaining times and completion flags before running each algorithm to ensure fair, identical starting states.
`printResults()`	Calculates Average Waiting Time and Average Turnaround Time, printing formatted result tables to stdout.
`printGanttChart()`	Iterates through the `LinkedList<GanttBlock>` to render a text-based visual execution timeline.
`openGUI()`	Cross-platform system call (`start` / `open` / `xdg-open`) that launches `index.html` in the user's web browser.

Cross-Platform System Command Strategy

```
#ifdef _WIN32 string command = "start \"\" \"" + fileName + "\""; // Windows #elif
```

SLIDE 4 OF 4

Main Execution Flow & Important Takeaways

Program Lifecycle and Evaluation Summary

Interactive Program Workflow (`main()`)

1. **Interactive User Input:** Prompts user for total process count, then loops to collect Process ID, Arrival Time, Burst Time, and Priority for each process.
2. **Time Quantum Configuration:** Prompts for Round Robin time quantum.
3. **Sequential Simulation Execution:** Automatically executes all 5 scheduling algorithms sequentially against identical input datasets.
4. **Console Output Generation:** Outputs per-process metric tables and Gantt charts to console.
5. **Web GUI Trigger:** Invokes `openGUI()` to auto-open `index.html` for interactive visual inspection.

Core Metrics & Formulas Summary

Turnaround Time (TAT)

`TAT = Completion Time - Arrival Time`

Total time taken from process submission to completion.

Waiting Time (WT)

`WT = Turnaround Time - Burst Time`

Total time process spent waiting in ready queue.

Key Highlights & Engineering Takeaways

- **Zero External Dependencies:** Written in pure standard C++ using custom template data structures (Lists, Queues, Stacks).
- **Fair Comparison:** Built-in state resetting ensures all algorithms are benchmarked on exact identical process states.
- **Preemptive Granularity:** Accurate tick-by-tick simulation guarantees high accuracy for SRTF and Round Robin metrics.
- **Hybrid Presentation:** Combines detailed CLI tabular execution output with web GUI integration.